

Application web de gestion de terrains de padel



Auteur	Grigorescu Marius - EPHEC
Version documentaire	2.0 : révision complète
Date du snapshot	7 août 2026
Révision du code	main : 9ce8b8f1cfd5

Contents

1. Contexte du projet	6
1.1. Objet du document	6
1.2. Présentation générale	6
1.3. Problématique métier couverte	6
1.4. Utilisateurs et rôles	6
1.5. Objectifs atteints par le MVP actuel	7
1.6. Fonctionnalités absentes du code actuel	7
1.7. Contraintes techniques structurantes	7
2. Analyse fonctionnelle	8
2.1. Vue générale de l'application	8
2.2. Accueil et données de démonstration	8
2.3. Inscription d'un joueur	8
2.4. Authentification joueur	9
2.5. Authentification administrateur	9
2.6. Consultation des disponibilités	9
2.7. Création d'un match	9
2.8. Match privé et invitations	10
2.9. Matches publics	10
2.10. Réservations et paiement d'une participation	10
2.11. Solde, dettes et historique	10
2.12. Traitement J-1	11
2.13. Traitement des échéances	11
2.14. Fermetures et remboursements	11
2.15. État opérationnel	11
2.16. Membres et statistiques administrateur	12
3. Synthèse des règles système	12
3.1. Paramètres métier centralisés	12
3.2. Sites, terrains, horaires et fermetures	13
3.3. Catégories et matricules	13
3.4. Disponibilités	13
3.5. Création d'un match	13
3.6. Participations	14
3.7. Invitations privées et passage public	14
3.8. Matches publics	14
3.9. Paiements et solde	14

3.10. Dettes	14
3.11. Pénalités.....	15
3.12. Cycle de vie d'un match	15
3.13. Remboursements lors d'une fermeture	15
3.14. Statistiques.....	15
3.15. Authentification et autorisation	15
3.16. Frontend et backend.....	16
4. Modèle de données	16
4.1. Vue générale	16
4.2. Sites, terrains et horaires	16
4.3. Fermetures.....	16
4.4. Membres et administrateurs	17
4.5. Matches et participations	17
4.6. Paiements, dettes et pénalités	17
4.7. Énumérations persistées.....	17
4.8. Cardinalités principales.....	17
5. Schéma relationnel SQL	18
5.1. Gestion du schéma.....	18
5.2. Relations SQL	18
5.3. Contraintes d'unicité.....	18
5.4. Contraintes de contrôle	19
5.5. Répartition des responsabilités	19
5.6. Verrouillage et concurrence	19
6. Architecture technique	19
6.1. Vue d'ensemble	19
6.2. Inventaire du code principal	19
6.3. Architecture backend.....	20
6.3.1. Controllers.....	20
6.3.2. Services	20
6.3.3. Repositories	20
6.3.4. Entités et DTO	20
6.3.5. Gestion des erreurs.....	20
6.3.6. Planification	21
6.4. Architecture frontend	21
6.4.1. Composants standalone et chargement paresseux.....	21
6.4.2. Façades et signaux	21

6.4.3. Services API et modèles	21
6.4.4. Contexte d'authentification	21
6.4.5. Guards et interceptor.....	21
6.5. Communication REST et OpenAPI.....	21
6.6. Frameworks et versions	22
7. Base de données et sécurité	22
7.1. Profil H2 par défaut.....	22
7.2. Profil PostgreSQL	22
7.3. Docker PostgreSQL.....	22
7.4. Utilisateurs et droits DB	22
7.5. Mots de passe	23
7.6. JWT	23
7.7. Chaîne de sécurité HTTP	23
7.8. Autorisation joueur	23
7.9. Autorisation administrateur	24
7.10. Session frontend	24
7.11. CORS et exposition locale	24
7.12. Points de durcissement identifiés dans le code.....	24
8. Tests	24
8.1. Organisation backend	24
8.2. Couverture backend visible.....	25
8.3. Configuration de test backend.....	25
8.4. Tests frontend	26
8.5. Compilation frontend.....	26
8.6. Cypress	26
8.7. GitHub Actions	26
9. Organisation Git et méthode de travail	26
9.1. Dépôt de référence	26
9.2. Historique observable	26
9.3. Contrôles de qualité versionnés	27
10. Exploitation du projet	27
10.1. Prérequis	27
10.2. Démarrage H2 par défaut	27
10.3. Démarrage du frontend	27
10.4. Démarrage PostgreSQL	27
10.5. Variables d'environnement utiles.....	28

10.6. Comptes de démonstration publiés.....	28
10.7. Données de démonstration	29
10.8. Commandes de validation	29
10.9. Ordre de démonstration conseillé.....	29
11. Limites du MVP	29
11.1. Limites fonctionnelles	29
11.2. Limites de sécurité	30
11.3. Limites de persistance et d'exploitation.....	30
11.4. Limites de performance et d'échelle	30
11.5. Limites de validation de session frontend	30
12. Brève conclusion	30
Annexes.....	31
Annexe A : Catalogue des endpoints REST	31
Annexe B : Routes Angular.....	32
Annexe C : Fichiers de traçabilité essentiels	33
Annexe D : Contrôle de version du présent document	33
Annexe E : schéma complet.....	34

1. Contexte du projet

1.1. Objet du document

Le projet consiste à développer une application web de gestion et de réservation de terrains de padel. Le présent cahier des charges décrit l'état effectivement implémenté de l'application web « Padel Marius ».

1.2. Présentation générale

L'application gère la réservation de terrains de padel sur plusieurs sites. Un site possède des terrains, des horaires annuels et peut être concerné par une fermeture globale ou locale. Les joueurs disposent de catégories de membre qui déterminent leur fenêtre de réservation et, pour la catégorie SITE, leur périmètre géographique.

Le cycle principal couvre la consultation des créneaux, la création d'un match privé ou public, la gestion des invitations et des inscriptions, le paiement sur un solde interne, le traitement des dettes et l'évolution automatique de l'état des matchs. Une interface d'administration permet de consulter l'état opérationnel, les membres et les statistiques, de créer des fermetures et de lancer manuellement les traitements réservés à l'administrateur global.

Le système est composé de trois parties strictement séparées :

- un frontend Angular exécuté dans le navigateur ;
- une API REST Spring Boot qui centralise la logique métier et la sécurité ;
- une base relationnelle H2 par défaut, ou PostgreSQL via un profil et Docker.

Le frontend ne contient aucune requête SQL et n'accède jamais directement à la base. Toute opération métier passe par les endpoints HTTP du backend.

1.3. Problématique métier couverte

La gestion de plusieurs sites impose de combiner des règles de disponibilité et des règles propres aux joueurs. Un créneau ne peut être proposé que si le site et le terrain sont actifs, si un horaire existe pour l'année demandée, si aucune fermeture applicable n'est enregistrée et si aucun match non annulé n'entre en conflit avec la durée du match et la pause obligatoire.

Le système doit en outre empêcher les incohérences suivantes : dépasser quatre participants actifs, inscrire deux fois le même membre, faire participer un même joueur à deux matchs qui se chevauchent, payer deux fois la même participation, réserver avec une dette ou une pénalité bloquante, ou créer un match sur un créneau qui n'est plus disponible.

Le modèle financier reste volontairement simple. Chaque joueur dispose d'un solde crédit interne. Le paiement d'une participation débite ce solde et confirme la participation. Lorsqu'un match arrive à son heure de début sans être entièrement payé, le montant restant devient une dette de l'organisateur. Un match créé en privé et encore incomplet à l'échéance peut également entraîner une pénalité de sept jours.

1.4. Utilisateurs et rôles

Deux familles d'utilisateurs sont implémentées.

- **Joueur** : il s'inscrit, se connecte avec son matricule et son mot de passe, consulte les disponibilités, crée un match, traite ses invitations, rejoint un match public, paie ses participations et ses dettes, puis consulte ses réservations, son solde et son historique.

- **Administrateur** : il se connecte avec un login et un mot de passe. Un administrateur GLOBAL accède à tous les sites et aux traitements manuels. Un administrateur SITE est limité à son site pour les membres, les statistiques, l'état opérationnel et les fermetures locales.

Les rôles sont contrôlés dans le backend à partir du JWT, puis revalidés contre l'utilisateur actif conservé en base. Les gardes Angular améliorent la navigation, mais ne constituent pas l'autorité de sécurité finale.

1.5. Objectifs atteints par le MVP actuel

Le code actuel met en œuvre les objectifs suivants :

- fournir une présentation de démonstration et des comptes de test lorsque le mode démo est actif ;
- inscrire un joueur avec un mot de passe chiffré et un matricule généré selon sa catégorie ;
- authentifier joueurs et administrateurs et leur délivrer un JWT signé ;
- proposer les créneaux disponibles par site et par date ;
- créer un match privé ou public avec une participation organisateur en attente de paiement ;
- inviter des joueurs à un match privé, confirmer l'invitation par paiement ou la décliner ;
- lister les matchs publics et calculer l'éligibilité du joueur avant l'inscription ;
- rejoindre et payer un match public dans une même transaction métier ;
- consulter et payer les participations depuis la vue des réservations ;
- calculer, afficher et régler les dettes de l'organisateur ;
- faire passer automatiquement les matchs de **A_VENIR** à **DEMARRE**, puis à **TERMINE** ;
- exécuter un traitement J-1 qui libère les invitations non payées et rend publics les matchs privés incomplets ;
- créer des fermetures globales ou locales, annuler les matchs futurs concernés et recréditer les paiements ;
- afficher l'état opérationnel des terrains, les membres et des statistiques agrégées ;
- exécuter le backend sur H2 ou PostgreSQL avec un schéma géré par Liquibase ;
- couvrir le backend, le frontend et deux parcours Cypress par des suites de tests dédiées ;
- exécuter des contrôles automatiques dans GitHub Actions lors des pull requests vers **main** et des pushes sur **main**.

1.6. Fonctionnalités absentes du code actuel

Le dépôt ne contient pas d'intégration avec un prestataire bancaire, de transfert d'argent réel, de messagerie électronique, de notification push, de WebSocket, d'application mobile native ni de récupération de mot de passe.

Il ne contient pas non plus d'interface complète de gestion des sites, terrains, horaires ou administrateurs. Les sites et terrains sont principalement consultés ; les données de démonstration sont préparées par migration ou par seeder. Les joueurs ne disposent pas d'un endpoint d'annulation volontaire d'un match ni d'un mécanisme de recharge de leur solde.

1.7. Contraintes techniques structurantes

Le backend utilise Java 21 et Spring Boot 4.0.6. Il respecte les couches **controller**, **service** et **repository**, complétées par les entités JPA, les DTO, la sécurité, les configurations et deux schedulers.

Le frontend utilise Angular 21.2.19, TypeScript 5.9, RxJS 7.8 et Angular Material/CDK 21.2.x. Les pages sont des composants standalone chargés paresseusement. La logique d'écran est majoritairement déplacée vers des services façades, tandis que les services API isolent les appels HTTP.

La persistance repose sur Liquibase et sur un schéma SQL commun à H2 et PostgreSQL. Hibernate est configuré en mode `validate` et ne crée pas automatiquement les tables. Le profil par défaut utilise une base H2 en mémoire ; le profil `postgres` utilise PostgreSQL 16 dans la configuration Docker fournie.

Traçabilité principale. Versions et dépendances : `backend/pom.xml` et `frontend/package.json`. Profils : `backend/src/main/resources/application.yml`, `application.properties` et `application-postgres.properties`.

2. Analyse fonctionnelle

2.1. Vue générale de l'application

L'application Angular présente une barre latérale commune. Les liens publics conduisent à l'accueil, à l'inscription joueur, à la connexion joueur et à la connexion administrateur. Les menus protégés apparaissent en fonction de l'identité conservée dans le contexte d'authentification.

Une session joueur expose les parcours « Organiser un match », « Rejoindre un match public », « Mes réservations », « Invitations reçues », « Mes dettes », « Historique des transactions » et « Mon solde ». Une session administrateur expose le dashboard, les membres, les statistiques, le planning hebdomadaire des terrains et les jours de fermeture. Les deux traitements manuels ne sont affichés que pour le rôle GLOBAL.

Le frontend conserve au maximum une identité active : enregistrer un joueur efface l'administrateur et inversement. Si les deux objets sont trouvés simultanément dans le stockage local, ils sont tous deux supprimés.

2.2. Accueil et données de démonstration

La page `/accueil` appelle `GET /api/demo/presentation`. Lorsque `padel.demo.enabled=true`, le backend retourne :

- la légende des préfixes G, S et L ;
- les sites actifs ;
- les comptes joueurs destinés à la démonstration ;
- les comptes administrateurs destinés à la démonstration.

Le contrôleur et son service sont conditionnels. Si le mode démo est désactivé, l'endpoint n'est pas enregistré. Les comptes et mots de passe affichés proviennent de `application.yml`, et non de valeurs codées directement dans le composant Angular.

2.3. Inscription d'un joueur

La route `/inscription-joueur` permet de saisir le nom, le prénom, la catégorie, le site de rattachement éventuel, le mot de passe et sa confirmation. Le backend exige un mot de passe de 12 à 72 caractères et vérifie l'égalité avec la confirmation.

Pour un membre SITE, un site actif est obligatoire. Pour GLOBAL et LIBRE, aucun site de rattachement n'est enregistré. Le matricule est généré à partir du préfixe de catégorie et du plus grand numéro existant, avec un point de départ à 1001. Le mot de passe est encodé avec BCrypt et le solde initial vaut 100,00 euros.

L'inscription est publique et retourne le DTO du membre créé, sans retourner le hash du mot de passe.

2.4. Authentification joueur

La page `/joueur` envoie le matricule et le mot de passe à `POST /api/auth/joueur`. Le backend recherche le membre, vérifie qu'il est actif et compare le mot de passe avec le hash BCrypt. L'échec d'identification utilise un message générique qui ne révèle pas si le matricule existe.

En cas de succès, la réponse contient l'identité métier, la catégorie, le site éventuel, le solde, le JWT et sa date d'expiration. Le frontend enregistre l'objet sous la clé `padel-joueur` et supprime une éventuelle session administrateur.

2.5. Authentification administrateur

La page `/admin/login` appelle `POST /api/auth/admin`. Le service vérifie le login, le mot de passe BCrypt et le caractère actif du compte. La réponse comprend le rôle GLOBAL ou SITE, le site éventuel, le JWT et son expiration.

Après connexion, le frontend stocke l'objet sous `padel-admin`, efface la session joueur et navigue vers `/admin/dashboard`.

2.6. Consultation des disponibilités

La route `/joueur/disponibilites` charge les sites actifs et les paramètres métier depuis le backend. L'utilisateur choisit un site et une date, ou utilise un des sept raccourcis de date proposés par le frontend.

Le backend vérifie successivement le site actif, les fermetures, l'horaire annuel de l'année, les terrains actifs et les matchs du jour. Il génère des créneaux de 90 minutes espacés de 15 minutes. Les matchs annulés ne bloquent pas les disponibilités.

Si la journée est fermée, la réponse signale la fermeture, son motif et une liste vide. Si le site ne possède aucun terrain actif, la réponse n'est pas considérée comme une fermeture mais ne contient aucun créneau.

Pour un membre SITE, le frontend désactive le passage vers la création d'un match en dehors du site de rattachement. Le backend réapplique cette règle lors de la création.

2.7. Création d'un match

Depuis un créneau disponible, le frontend transmet le terrain et la date de début à `/joueur/creer-match` au moyen des paramètres d'URL. Le joueur choisit le mode PRIVE ou PUBLIC. Son matricule est prérempli à partir de la session.

`POST /api/matches` verrouille le terrain et l'organisateur. Le service vérifie notamment :

- l'activité du site, du terrain et du membre ;
- une date de début strictement future ;
- la catégorie et la fenêtre de réservation ;
- l'absence de dette ouverte et de pénalité active ;
- l'absence d'une autre participation organisateur non payée ;
- l'existence exacte du créneau dans les disponibilités recalculées ;
- l'absence de conflit horaire du joueur, pause de 15 minutes comprise.

Le match est créé pour 90 minutes, au prix total de 60,00 euros, avec l'état `A_VENIR`. Sa visibilité initiale correspond au mode de création. Une participation ORGANISATEUR est créée avec le statut `EN_ATTENTE_PAIEMENT`. Le paiement n'est donc pas implicite.

2.8. Match privé et invitations

Après la création d'un match privé, l'organisateur peut inviter un joueur par matricule. Seul l'organisateur authentifié peut appeler l'endpoint, et les invitations ne sont possibles que si le match a été créé en mode PRIVE et est encore visible comme PRIVE.

Une invitation crée une participation JOUEUR de mode `INVITATION_PRIVEE` et de statut `EN_ATTENTE_PAIEMENT`. Elle occupe une place active tant qu'elle n'est pas libérée. Le service refuse un membre inactif, un cinquième participant, un doublon ou un conflit horaire.

Le joueur invité consulte `/joueur/invitations-recues`. Il peut :

- confirmer et payer, ce qui crée un paiement et passe la participation à `CONFIRMEE` ;
- décliner, ce qui passe la participation à `LIBEREE` et enregistre la date de libération.

Un badge dans la navigation affiche le nombre d'invitations encore en attente. Ce compteur est chargé depuis l'API et mis à jour après une action réussie.

2.9. Matches publics

La route `/joueur/matches-publics` recherche les matchs publics d'un site et d'une date. Le backend ne retient que les matchs `A_VENIR`, sur un site et un terrain actifs, avec au moins une place disponible.

Pour chaque match, la réponse indique le nombre de participants actifs, les places restantes, le prix, le montant standard, l'éligibilité du joueur et, le cas échéant, le motif de refus. Le calcul tient compte d'une participation existante, d'un conflit horaire et des règles de catégorie.

L'action « rejoindre et payer » crée la participation publique puis effectue le paiement dans le même service transactionnel. La réponse retourne les identifiants techniques nécessaires au frontend, le montant total débité, le statut confirmé et le nouveau solde.

2.10. Réservations et paiement d'une participation

La page `/joueur/mes-reservations` liste les participations non libérées du joueur, triées par date de début. Elle affiche le site et le numéro du terrain, les dates, le rôle, le mode d'entrée, le statut de la participation, le mode et la visibilité du match, ainsi que l'état du cycle.

Une participation en attente peut être payée depuis cette page. Le montant transmis par le frontend est 15,00 euros et le backend exige exactement cette valeur. Le service refuse les doubles paiements, les participations libérées ou déjà confirmées, les membres inactifs, les matchs qui ont commencé et les soldes insuffisants.

Le paiement débite le solde interne, crée une ligne `paiement` de nature `PARTICIPATION` et confirme la participation.

2.11. Solde, dettes et historique

La page `/joueur/mon-solde` consulte le solde courant. La page `/joueur/mes-dettes` liste les dettes `OUVERTE` dont le joueur est responsable et préremplit, pour chacune, le montant restant.

Le paiement direct d'une dette exige exactement le montant restant ; aucun paiement partiel n'est prévu. Le solde est débité, la dette passe à `REGLEE` et un paiement de nature `REGLEMENT_DETTE` est créé.

Le paiement d'une participation possède en plus un comportement de régularisation : avant de confirmer la participation courante, le service actualise les dettes de l'organisateur et ajoute au débit

toutes ses autres dettes ouvertes. Ces dettes sont réglées par des paiements séparés dans la même transaction. Le solde doit donc couvrir la participation et les dettes concernées.

La page [/joueur/historique-transactions](#) retourne les paiements du joueur par date décroissante, avec la nature, le montant, le statut et les liens éventuels vers la participation, la dette et le match.

2.12. Traitement J-1

Le traitement J-1 prend une date de traitement et analyse les matchs du lendemain. Pour chaque match [A_VENIR](#), il libère les participations JOUEUR encore en attente de paiement. Si un match actuellement PRIVE reste incomplet après cette libération, il devient PUBLIC et reçoit une date de passage public.

Le traitement est planifié par défaut avec un délai initial de 30 secondes puis un délai fixe d'une heure. L'administrateur global peut également le lancer manuellement depuis [/admin/traitement-veille](#).

La réponse indique la date traitée, la date des matchs, le nombre de matchs analysés, le nombre de passages en public et le nombre de participations libérées.

2.13. Traitement des échéances

Le traitement d'échéance recherche les matchs [A_VENIR](#) dont l'heure de début est atteinte. Il calcule la dette éventuelle, crée une pénalité si un match créé en privé est incomplet, puis passe le match à [DEMARRE](#).

Dans le même passage, il recherche les matchs [DEMARRE](#) dont l'heure de fin est atteinte et les passe à [TERMINE](#). La réponse restitue les matchs analysés, démarrés, terminés et les dettes créées.

Le scheduler est actif par défaut après un délai initial de 60 secondes, puis toutes les 60 secondes. Le lancement manuel via [/admin/traitement-echeance](#) est réservé à l'administrateur global.

2.14. Fermetures et remboursements

La page [/admin/fermetures](#) permet de créer une fermeture GLOBALE ou LOCALE. Un administrateur SITE ne peut créer qu'une fermeture locale pour son propre site. Un administrateur GLOBAL peut choisir la portée et le site.

Le service refuse les combinaisons portée/site incohérentes et les doublons. Il empêche également de transformer une journée déjà couverte par des fermetures locales en fermeture globale, afin d'éviter une superposition ambiguë.

Après création, tous les matchs [A_VENIR](#) concernés à cette date sont verrouillés et passent à [ANNULE](#). Les paiements de participation encore [PAYE](#) sont recredités sur les soldes correspondants et passent à [ANNULE](#). La réponse donne le nombre de matchs annulés, le nombre de remboursements et le montant total recredité.

2.15. État opérationnel

La page [/admin/etat-operationnel](#) est désormais une vue d'occupation hebdomadaire.

L'administrateur choisit une date quelconque ; le backend la ramène au lundi de la semaine correspondante et retourne les sept journées jusqu'au dimanche. Des boutons permettent ensuite d'afficher la semaine précédente, la semaine courante ou la semaine suivante.

Le planning présente une ligne par terrain et une colonne par jour. Il inclut tous les terrains du site, y compris les terrains inactifs, et distingue INACTIF, FERME, RESERVE et DISPONIBLE. Pour chaque réservation, l'écran affiche l'heure de début et de fin, le nombre de participants sur quatre, l'état du

cycle et la visibilité. Les matchs annulés restent visibles avec un traitement distinct et ne sont pas comptés parmi les réservations actives.

Trois indicateurs synthétisent le nombre de terrains, le nombre de réservations actives de la semaine et le nombre de jours fermés. Une fermeture globale ou locale applicable affiche son motif. Un site ne contenant aucun terrain produit un état vide explicite.

L'API quotidienne GET /api/admin/etat-operationnel reste présente. L'écran Angular consomme toutefois GET /api/admin/etat-operationnel/semaine. Ce nouvel endpoint refuse un site inactif avec une erreur métier 409. Un administrateur SITE ne peut consulter que le site rattaché à son compte ; un administrateur GLOBAL sélectionne le site.

2.16. Membres et statistiques administrateur

La page /admin/membres liste les membres et leur solde. L'administrateur GLOBAL peut afficher tous les membres ou sélectionner un site. L'administrateur SITE est limité à son site.

La page /admin/statistiques calcule les indicateurs sur une période inclusive :

- nombre total de matchs non annulés ;
- matchs à venir et terminés ;
- paiements payés et chiffre d'affaires ;
- dettes ouvertes et montant restant ;
- participations actives ;
- capacité théorique à quatre joueurs par match ;
- taux de remplissage en pourcentage.

Les paiements liés à des matchs annulés sont exclus. Le filtre de site est facultatif uniquement pour l'administrateur global.

3. Synthèse des règles système

3.1. Paramètres métier centralisés

Les paramètres fondamentaux sont définis une seule fois dans ReglesMetier et exposés au frontend par GET /api/parametres-metier.

Paramètre	Valeur actuelle	Effet dans le code
Durée d'un match	90 minutes	Calcul de la fin, créneaux et chevauchements
Pause obligatoire	15 minutes	Séparation des matchs d'un terrain et d'un joueur
Capacité maximale	4 joueurs	Refus du cinquième participant et statistiques
Prix total	60,00 €	Référence du match et calcul de dette
Participation standard	15,00 €	Montant exigé pour chaque paiement de participation
Solde initial	100,00 €	Crédit attribué à l'inscription
Pénalité	7 jours	Blocage après match privé incomplet
Fenêtre GLOBAL	21 jours	Date maximale de réservation
Fenêtre SITE	14 jours	Date maximale, uniquement sur le site rattaché
Fenêtre LIBRE	5 jours	Date maximale de réservation

3.2. Sites, terrains, horaires et fermetures

- Seuls les sites actifs sont retournés aux parcours joueurs.
- Seuls les terrains actifs appartenant à un site actif sont retournés par l'endpoint public des terrains.
- Un horaire annuel unique doit exister pour le couple site/année.
- L'heure de fin doit être postérieure à l'heure de début.
- Une fermeture locale applicable est recherchée avant la fermeture globale dans le calcul des disponibilités.
- Une fermeture globale ne possède pas de site ; une fermeture locale doit en posséder un.
- Une fermeture annule uniquement les matchs [A_VENIR](#) de la date et du périmètre concernés.
- Le planning hebdomadaire transforme toute date reçue en une période lundi-dimanche de sept jours.
- Le planning hebdomadaire n'est pas disponible pour un site inactif.
- La vue administrative conserve les terrains inactifs et leur attribue l'état INACTIF ; sur un terrain actif, une fermeture prévaut sur une réservation, puis une réservation non annulée prévaut sur la disponibilité.

3.3. Catégories et matricules

Catégorie	Préfixe vérifié	Sites autorisés	Fenêtre maximale
GLOBAL	G	Tous les sites actifs	J+21
SITE	S	Site de rattachement uniquement	J+14
LIBRE	L	Tous les sites actifs	J+5

Le préfixe est vérifié lors de l'application des règles de réservation. L'inscription génère elle-même un matricule conforme à la catégorie.

La date ne peut pas être postérieure à la fenêtre maximale. La création d'un match impose en plus une date et une heure strictement futures.

3.4. Disponibilités

Les créneaux sont générés à partir de l'heure d'ouverture. Chaque itération ajoute 90 minutes de match et 15 minutes de pause. Un créneau n'est ajouté que si sa fin ne dépasse pas l'heure de fermeture.

Un match existant bloque le créneau sauf s'il est [ANNULE](#). Le conflit intègre la pause avant et après : le nouveau match doit finir au moins 15 minutes avant le suivant ou commencer au moins 15 minutes après la fin du précédent.

La création ne se contente pas de faire confiance au créneau sélectionné par le navigateur. Elle recharge les disponibilités sous verrou et exige une correspondance exacte du terrain, du début et de la fin.

3.5. Création d'un match

L'organisateur doit être actif. Il ne peut pas créer un match dans le passé, avec une dette ouverte, avec une pénalité active non expirée ou tant qu'une autre participation ORGANISATEUR reste en attente de paiement sur un match non annulé.

Il ne peut pas non plus participer à un autre match qui chevauche le nouveau créneau, pause comprise. Ces vérifications portent sur les participations non libérées.

À la création :

- `etatCycle = A_VENIR` ;
- `prixTotal = 60.00` ;
- `dateHeureFin = dateHeureDebut + 90 minutes` ;
- `visibiliteCourante = PRIVE` ou `PUBLIC` selon le mode ;
- la participation de l'organisateur est `EN_ATTENTE_PAIEMENT`.

3.6. Participations

Le système distingue les rôles ORGANISATEUR et JOUEUR, les modes d'entrée CREATION, INVITATION_PRIVEE et INSCRIPTION_PUBLIQUE, ainsi que les statuts `EN_ATTENTE_PAIEMENT`, `CONFIRMEE` et `LIBEREE`.

Une participation en attente occupe une place. Seule une participation libérée est exclue du comptage des participants actifs. La contrainte SQL interdit deux lignes pour le même couple match/membre.

Le contrôle de catégorie est appliqué à la création du match et à l'inscription publique. L'ajout d'un invité privé ne rappelle pas la fenêtre de catégorie ; il vérifie le match privé, l'activité du terrain et du site, la capacité, le doublon et le conflit horaire. Cette distinction correspond exactement aux appels présents dans les services actuels.

3.7. Invitations privées et passage public

Seul l'organisateur d'un match créé en privé peut inviter, tant que la visibilité courante reste PRIVE. Une invitation en attente peut être payée ou déclinée.

Lors du traitement J-1, les participations JOUEUR non payées sont libérées. Si le match reste incomplet, sa visibilité passe à PUBLIC. Son `modeCreation` demeure toutefois PRIVE, ce qui conserve l'origine du match pour le calcul ultérieur de la pénalité.

3.8. Matches publics

La liste publique ne contient que les matchs à venir de la date, sur un terrain et un site actifs, avec au moins une place. Elle ne masque pas automatiquement un match auquel le joueur participe déjà ou pour lequel il est inéligible : elle retourne `peutRejoindre=false` et un motif explicite.

L'inscription et le paiement public sont exécutés dans une transaction englobante. Les validations du service de participation et du service de paiement restent donc appliquées au moment de l'action.

3.9. Paiements et solde

Le paiement est un débit de crédit interne et ne correspond pas à une transaction bancaire. Le montant d'une participation doit être exactement 15,00 euros. Un paiement réussi confirme la participation.

Une participation ne peut être payée qu'une fois. La base impose également l'unicité de `participation_id` dans `paiement`.

Lors d'un paiement de participation, les autres dettes ouvertes de l'organisateur sont ajoutées au débit total, puis marquées réglées avec leurs propres lignes de paiement. Si le solde total est insuffisant, l'ensemble de l'opération échoue transactionnellement.

3.10. Dettes

Une dette peut être calculée à partir de l'heure de début d'un match non annulé. Son montant restant est :

`prix total du match - somme des paiements de participation au statut PAYE.`

Une seule dette est autorisée par match. Le membre responsable est l'organisateur. Une dette entièrement couverte devient `REGLEE` ; sinon elle reste `OUVERTE`.

Le paiement direct doit égaliser exactement le montant restant. Le code n'implémente pas de paiement partiel.

3.11. Pénalités

À l'échéance, un match dont le `modeCreation` est `PRIVE` et qui possède moins de quatre participations confirmées génère une pénalité `RESERVATION_PRIVEE_INCOMPLETE` pour l'organisateur. Sa durée est de sept jours.

Une seule pénalité est créée par match source. Lors d'une nouvelle réservation, les pénalités actives dont la date de fin est atteinte passent à `TERMINEE`. Une autre pénalité active bloque la création.

3.12. Cycle de vie d'un match

L'état normal évolue de `A_VENIR` vers `DEMARRE` à l'heure de début, puis vers `TERMINE` lorsque l'heure de fin est atteinte. Une fermeture administrative peut faire passer un match encore à venir à `ANNULE`.

La visibilité `PRIVE/PUBLIC` est indépendante de l'état du cycle. Le traitement J-1 peut donc modifier la visibilité sans changer `A_VENIR`.

3.13. Remboursements lors d'une fermeture

Les paiements de participation `PAYE` d'un match annulé sont recrédités individuellement. Chaque paiement passe à `ANNULE`. Les membres concernés sont verrouillés dans l'ordre de leur identifiant afin de limiter les conflits concurrents.

Les statistiques excluent ensuite les paiements liés à un match annulé.

3.14. Statistiques

La période est inclusive des deux dates. Une date de fin antérieure à la date de début est refusée. Les matchs annulés sont exclus.

Le taux de remplissage est calculé comme suit :

`participations actives / (nombre de matchs × 4) × 100`, avec deux décimales.

Le chiffre d'affaires est la somme des paiements au statut `PAYE` dans la période, après exclusion des paiements liés à des matchs annulés.

3.15. Authentification et autorisation

- Les mots de passe sont comparés avec BCrypt.
- Les JWT sont signés en HMAC-SHA256 et expirent après 60 minutes.
- Le backend est stateless : aucune session HTTP n'est créée.
- Les endpoints administrateur exigent `ROLE_ADMIN`; les endpoints joueur exigent `ROLE_JOUEUR`.
- Les règles de propriétaire et de périmètre sont appliquées par `@PreAuthorize` et par les services d'autorisation.
- L'utilisateur et son caractère actif sont rechargés depuis la base pour les décisions sensibles.
- Les erreurs 401 et 403 sont rendues au même format JSON que les autres erreurs API.

3.16. Frontend et backend

Les gardes Angular redirigent un utilisateur non connecté vers la page de connexion appropriée. Le garde global renvoie un administrateur SITE vers son dashboard.

L'interceptor ajoute le JWT administrateur aux URL contenant `/api/admin/` et le JWT joueur aux autres appels `/api/`. Le backend demeure responsable de l'autorisation, y compris lorsque l'URL est appelée hors du frontend.

Les composants affichent l'état exposé par les façades. Les façades gèrent les signaux, les validations d'interface, les appels asynchrones, les messages et la navigation. Les services API contiennent les URLs et les types HTTP.

4. Modèle de données

4.1. Vue générale

Le domaine persistant comprend onze entités JPA correspondant aux onze tables créées par Liquibase. Les identifiants sont des `BIGINT` générés par identité. Les énumérations sont enregistrées sous forme de chaînes afin de conserver des valeurs lisibles dans la base.

Entité / table	Rôle principal	Relations structurantes
Site / site	Centre de padel	Parent des terrains, horaires, fermetures locales, membres SITE et admins SITE
Terrain / terrain	Terrain numéroté d'un site	Plusieurs terrains pour un site ; parent des matchs
HoraireAnnuelSite horaire_annuel_site	/ Plage de réservation par année	Une ligne par site et année
Fermeture / fermeture	Journée fermée globale ou locale	Site nul en global, obligatoire en local
Membre / membre	Joueur et solde	Site facultatif ; parent des participations, paiements, dettes responsables et pénalités
Administrateur administrateur	/ Compte d'administration	Site facultatif selon le rôle
PadelMatch / padel_match	Réservation d'un terrain	Un terrain ; plusieurs participations ; au plus une dette
Participation / participation	Place d'un membre dans un match	Un match et un membre
Paieement / paieement	Débit interne	Cible exclusive : participation ou dette
Dette / dette	Reste dû pour un match	Un match unique et un membre responsable
Penalite / penalite	Blocage temporel	Un membre et un match source

4.2. Sites, terrains et horaires

Un site possède un code unique, un nom, une adresse et un indicateur d'activité. Un terrain appartient obligatoirement à un site ; son numéro est unique à l'intérieur de ce site.

L'horaire annuel associe une année civile, une heure de début et une heure de fin à un site. L'unicité du couple site/année empêche deux horaires concurrents pour la même période.

4.3. Fermetures

Une fermeture stocke une date, une portée, un motif facultatif et un site facultatif. La contrainte SQL `ck_fermeture_portee_site` impose les deux configurations valides : GLOBALE sans site ou LOCALE avec site.

Le service complète cette contrainte en empêchant les doublons et les superpositions globales/locales ambiguës.

4.4. Membres et administrateurs

Le membre stocke le matricule unique, l'identité, la catégorie, le site éventuel, l'activité, le hash du mot de passe et le solde crédit en `DECIMAL(10,2)`.

L'administrateur stocke un login unique, l'identité, le hash du mot de passe, le rôle, le site éventuel et l'activité. Le schéma autorise techniquement un site nul ou non nul ; la cohérence entre rôle et site est appliquée par les données et les services d'autorisation.

4.5. Matches et participations

Le match conserve le terrain, le début, la fin, le mode de création, la visibilité courante, le prix total, les dates de création et de passage public ainsi que l'état du cycle.

La participation relie un membre au match et conserve son rôle, son mode d'entrée, son statut et les dates d'affectation, confirmation et libération. L'unicité (`match_id`, `membre_id`) prévient un doublon durable.

4.6. Paiements, dettes et pénalités

Un paiement possède une nature, un montant, une date, un statut et un membre. Il cible soit une participation, soit une dette. La contrainte `ck_paiement_cible` refuse l'absence de cible et la double cible.

La dette est unique par match et conserve les montants initial et restant, le responsable, les dates et le statut. La pénalité conserve le type, le motif, les dates, le statut, le membre et le match source.

4.7. Énumérations persistées

Domaine	Valeurs présentes dans le code
Catégorie membre	GLOBAL, SITE, LIBRE
Rôle administrateur	GLOBAL, SITE
Mode de création	PRIVE, PUBLIC
Visibilité	PRIVE, PUBLIC
Cycle du match	A_VENIR, DEMARRE, TERMINE, ANNULE
Rôle participation	ORGANISATEUR, JOUEUR
Mode d'entrée	CREATION, INVITATION_PRIVEE, INSCRIPTION_PUBLIQUE
Statut participation	EN_ATTENTE_PAIEMENT, CONFIRMEE, LIBREE
Nature paiement	PARTICIPATION, REGLEMENT_DETTE
Statut paiement	EN_ATTENTE, PAYE, ANNULE
Statut dette	OUVERTE, REGLEE
Portée fermeture	GLOBALE, LOCALE
Statut pénalité	ACTIVE, TERMINEE, ANNULEE

4.8. Cardinalités principales

- Un site possède zéro à plusieurs terrains et horaires annuels.
- Un terrain appartient à un seul site et reçoit zéro à plusieurs matches.
- Un membre possède zéro à plusieurs participations et paiements.

- Un match possède plusieurs participations, au plus une dette et peut être la source de pénalités.
- Une participation appartient à un seul membre et à un seul match.
- Une dette appartient à un seul match et à un seul membre responsable.
- Un paiement cible exactement une participation ou une dette.

Traçabilité au code. Entités : `backend/src/main/java/com/padelMarius/backend/entity`. Schéma : `backend/src/main/resources/db/changelog/changes/001-create-initial-schema.sql`.

5. Schéma relationnel SQL

5.1. Gestion du schéma

Le schéma n'est pas créé par Hibernate. Liquibase exécute le fichier maître `db.changelog-master.yaml`, qui inclut la migration `001-create-initial-schema.sql` puis les données H2 de démonstration. Hibernate utilise `ddl-auto=validate` pour vérifier la compatibilité entre les entités et les tables.

Le même script initial crée les onze tables sur H2 et PostgreSQL. Les types utilisés restent communs : identités `BIGINT`, chaînes `VARCHAR`, booléens, dates, heures, timestamps et montants `DECIMAL(10,2)`.

5.2. Relations SQL

Table enfant	Clé étrangère	Table parent	Caractère
terrain	site_id	site	Obligatoire
horaire_annuel_site	site_id	site	Obligatoire
fermeture	site_id	site	Selon la portée
membre	site_rattachement_id	site	Facultatif
administrateur	site_id	site	Facultatif
padel_match	terrain_id	terrain	Obligatoire
participation	match_id, membre_id	padel_match, membre	Obligatoires
dette	match_id, membre_responsable_id	padel_match, membre	Obligatoires
penalite	membre_id, match_source_id	membre, padel_match	Obligatoires
paiement	membre_id	membre	Obligatoire
paiement	participation_id ou dette_id	participation ou dette	Une cible exactement

Le script ne déclare pas de suppression en cascade. Une suppression de données parentes doit donc respecter les clés étrangères ou être réalisée dans un ordre contrôlé.

5.3. Contraintes d'unicité

Le schéma impose notamment :

- `site.code` unique ;
- `(terrain.site_id, terrain.numero)` unique ;
- `(horaire_annuel_site.site_id, annee_civile)` unique ;
- `membre.matricule` unique ;
- `administrateur.email_ou_login` unique ;
- `(participation.match_id, participation.membre_id)` unique ;

- une dette par match ;
- un paiement par participation ;
- un paiement par dette.

Ces contraintes constituent une seconde ligne de défense après les vérifications des services.

5.4. Contraintes de contrôle

Deux contraintes `CHECK` sont présentes :

- la portée d'une fermeture doit correspondre à la présence ou l'absence du site ;
- un paiement doit cibler exclusivement une participation ou une dette.

Les valeurs possibles des énumérations ne sont pas limitées par des contraintes SQL dédiées ; elles sont contrôlées par les énumérations Java et la validation JPA.

5.5. Répartition des responsabilités

La base garantit les clés, les relations, la nullabilité, les longueurs, les montants à deux décimales, les unicités et les deux cohérences structurelles précédentes.

Le backend garantit les règles évolutives : fenêtres de réservation, activité des sites, durée, pause, capacité, conflits horaires, propriété des actions, paiement exact, solde suffisant, calcul de dette, cycle des matchs, remboursements et périmètre administrateur.

La capacité à quatre joueurs et l'absence de chevauchement ne sont pas des contraintes SQL. Elles sont protégées par les transactions, les contrôles de service et les verrous pessimistes.

5.6. Verrouillage et concurrence

Les repositories fournissent des lectures `PESSIMISTIC_WRITE` pour les terrains, matchs, membres, participations et dettes concernés par les écritures sensibles. Les traitements de veille, d'échéance et de fermeture récupèrent les matchs dans l'ordre de leur identifiant.

Le remboursement verrouille également les membres dans un ordre stable. Ces choix réduisent les risques de double paiement, de double inscription et de traitement concurrent du même match.

6. Architecture technique

6.1. Vue d'ensemble

L'architecture suit un flux unidirectionnel : le navigateur appelle l'API REST ; l'API applique la sécurité et la logique ; les repositories utilisent JPA ; Liquibase maintient le schéma relationnel.

Le frontend et le backend sont deux projets autonomes. Le proxy de développement Angular redirige `/api` vers le port 8080. En parallèle, le backend limite CORS à `http://localhost:4200` pour les appels locaux directs.

6.2. Inventaire du code principal

Élément	Nombre
Classes Java sous <code>src/main</code>	146
Controllers REST	21
Services métier et d'autorisation	24
Repositories Spring Data	11
Fichiers d'entités et d'énumérations	24

Élément	Nombre
DTO Java	45
Schedulers	2
Pages Angular	19
Services API Angular	20
Services façades Angular	18
Modèles TypeScript	20
Guards Angular	3
Endpoints REST applicatifs	30

6.3. Architecture backend

6.3.1. Controllers

Les controllers traduisent HTTP en appels de service. Ils portent les chemins, les paramètres, les DTO, les statuts 200/201 et les expressions `@PreAuthorize`. Ils ne contiennent pas le calcul métier principal.

Les 29 endpoints sont répartis entre l'authentification, les référentiels publics, les parcours joueurs et l'administration. Le catalogue complet figure en annexe A.

6.3.2. Services

Les services appliquent les transactions et la logique. Les responsabilités sont séparées : création, disponibilités, participations, invitations, paiements, dettes, fermetures, statistiques, état opérationnel, inscription, authentification, règles de membre et traitements planifiés.

Les méthodes d'écriture sont annotées `@Transactional`. Les consultations utilisent généralement `readOnly=true`.

6.3.3. Repositories

Les onze interfaces héritent de `JpaRepository`. Elles combinent des méthodes dérivées et des requêtes JPQL. Les opérations concurrentes utilisent `@Lock(PESSIMISTIC_WRITE)`.

6.3.4. Entités et DTO

Les entités représentent la persistance et ne sont pas exposées directement comme contrat HTTP. Les records DTO séparent les requêtes et les réponses, appliquent Jakarta Validation et évitent notamment d'exposer les hashes de mots de passe.

6.3.5. Gestion des erreurs

`ApiExceptionHandler` normalise les erreurs dans un objet `{ code, message }`.

Situation	HTTP	Code API
Ressource absente	404	RESSOURCE_INTROUVABLE
Conflit ou règle métier	409	CONFIGURATION_METIER_INVALIDE
Authentification invalide	401	AUTHENTIFICATION_INVALIDE
Autorisation refusée	403	ACCES_REFUSE
Validation de champ	400	VALIDATION_INVALIDE
Paramètre absent ou invalide	400	REQUETE_INVALIDE
JSON illisible	400	JSON_INVALIDE

Les erreurs émises directement par Spring Security sont sérialisées par `SecurityErrorWriter` au même format.

6.3.6. Planification

`PlanificationConfig` active le scheduling. Deux composants conditionnels appellent les services de veille et d'échéance. Leur activation, délai initial et intervalle sont configurables par propriétés et variables d'environnement.

6.4. Architecture frontend

6.4.1. Composants standalone et chargement paresseux

Les 19 pages sont déclarées comme composants standalone. `app.routes.ts` les charge par `loadComponent`, ce qui produit des chunks paresseux distincts lors de la compilation.

Les templates utilisent la syntaxe Angular moderne `@if` et `@for`. Angular Material fournit notamment les boutons, cartes, formulaires, tableaux, badges et la barre d'outils.

6.4.2. Façades et signaux

La plupart des pages fournissent une façade locale. Celle-ci expose des signaux en lecture seule, prépare l'état initial, valide les champs d'interface, appelle les services API et transforme les réponses en messages et états d'écran.

Les changements de session sont observés avec `effect`. Les opérations asynchrones utilisent RxJS (`tap`, `catchError`, `finalize`, `timeout`, `takeUntil`) afin de gérer le chargement et d'éviter qu'une réponse d'une ancienne session ne modifie la nouvelle.

6.4.3. Services API et modèles

Chaque domaine possède un service HTTP typé : authentification, sites, terrains, paramètres, disponibilités, matchs, invitations, paiements, dettes, réservations et administration. Les interfaces TypeScript reprennent les DTO du backend.

6.4.4. Contexte d'authentification

`AuthContextService` contient des signaux joueur et administrateur. Il lit et écrit `localStorage`, élimine le JSON illisible, refuse une session sans expiration valide et écoute l'événement `storage` pour synchroniser les onglets.

Il n'exécute toutefois pas de minuterie d'expiration pendant une page déjà ouverte ; un token expiré sera finalement refusé par le backend.

6.4.5. Guards et interceptor

`joueurGuard` protège les huit routes joueur. `adminGuard` protège le dashboard et les pages administratives ordinaires. `adminGlobalGuard` protège les deux traitements manuels.

L'interceptor fonctionnel sélectionne le token selon l'URL. Il ajoute `Authorization: Bearer ...` aux appels `/api/` lorsqu'un token approprié existe.

6.5. Communication REST et OpenAPI

Le backend expose la documentation OpenAPI 3 via `/v3/api-docs` et l'interface Swagger via `/swagger-ui.html`. La configuration déclare le serveur local `http://localhost:8080` et la version fonctionnelle 1.0.0.

Les appels Angular utilisent une base d'URL vide en développement. Le proxy `proxy.conf.json` transmet `/api` vers `http://localhost:8080`, sans TLS local.

6.6. Frameworks et versions

Couche	Technologies et versions déclarées
Backend	Java 21, Spring Boot 4.0.6, Spring MVC, Data JPA, Security, Validation
Documentation	springdoc-openapi 3.0.3
Migration	Liquibase via le starter Spring Boot
Base	H2 runtime, PostgreSQL runtime, PostgreSQL 16-alpine dans Docker
Tests backend	JUnit/Spring test, Spring Security test, Testcontainers 2.0.5
Frontend	Angular 21.2.19, TypeScript 5.9.2, RxJS 7.8
Interface	Angular Material et CDK 21.2.x
Tests frontend	Vitest 4.x, jsdom 28, Cypress 15.18.1
Outils Node	Node ^22.12.0 ou ^24.0.0, npm 11.6.0

7. Base de données et sécurité

7.1. Profil H2 par défaut

Sans profil supplémentaire, le backend utilise `jdbc:h2:mem:padeldb`, l'utilisateur `sa` et un mot de passe vide. La base est en mémoire et disparaît lorsque le processus s'arrête.

Liquibase crée le schéma et charge les données de démonstration H2 avec le contexte `demo`. Les dates sont relatives à `CURRENT_DATE`, ce qui maintient les scénarios utilisables dans le temps.

La dépendance de console H2 et l'autorisation Spring Security pour `/h2-console/**` sont présentes. Le profil PostgreSQL désactive explicitement cette console ; la configuration H2 de base ne déclare pas explicitement la propriété `spring.h2.console.enabled`.

7.2. Profil PostgreSQL

Le profil `postgres` utilise par défaut `jdbc:postgresql://localhost:5432/padel_db`. Les identifiants peuvent être remplacés par variables d'environnement.

Liquibase se connecte avec `padel_migration`, tandis que JPA se connecte avec `padel_app`. `PostgresDemoDataSeeder`, actif sous le profil `postgres`, insère ou met à jour les données de démonstration et synchronise les séquences. Ce composant n'est pas conditionné par `padel.demo.enabled` dans le code actuel.

7.3. Docker PostgreSQL

`docker-compose.yml` lance `postgres:16-alpine` sur le port 5432, avec une base `padel_db`, un administrateur technique `padel_admin`, un volume persistant et un healthcheck `pg_isready`.

Le script monté dans `/docker-entrypoint-initdb.d` est exécuté lors du premier démarrage du volume. Il crée les rôles applicatifs et limite les droits publics sur le schéma `public`.

7.4. Utilisateurs et droits DB

Rôle PostgreSQL	Usage	Droits principaux
<code>padel_admin</code>	Initialisation du conteneur	Propriétaire initial de la base, compte Docker
<code>padel_migration</code>	Liquibase	Connexion, usage et création dans le schéma public
<code>padel_app</code>	Backend JPA	SELECT, INSERT, UPDATE, DELETE ; usage des séquences ; pas de CREATE de table

Rôle PostgreSQL	Usage	Droits principaux
<code>padel_readonly</code>	Consultation technique	SELECT sur les tables ; lecture des séquences ; pas d'écriture

Les privilèges par défaut garantissent que les objets créés ultérieurement par `padel_migration` restent accessibles à `padel_app` et `padel_readonly` selon leur rôle.

7.5. Mots de passe

`PasswordEncodingConfig` fournit un `BCryptPasswordEncoder`. Les inscriptions et les seeders PostgreSQL encodent les mots de passe avant persistance. Les données H2 contiennent des hashes BCrypt pré-calculés.

Les réponses d'authentification et les DTO de membre n'exposent jamais `motDePasseHash`. Les erreurs de login joueur et administrateur utilisent le même message générique.

7.6. JWT

Le projet construit ses JWT dans `JwtService`, sans bibliothèque JWT externe. Le header annonce `HS256` et `JWT`. La signature utilise `HmacSHA256` et un encodage Base64 URL sans padding.

Le payload contient :

- `sub` : matricule joueur ou login administrateur ;
- `typeUtilisateur` : `JOUEUR` ou `ADMIN` ;
- `role` : catégorie joueur ou rôle administrateur ;
- `siteId` : site éventuel ;
- `iat` : date de création en secondes Unix ;
- `exp` : date d'expiration en secondes Unix.

La signature est comparée avec `MessageDigest.isEqual`. Un token mal formé, mal signé ou expiré produit une erreur 401. La durée actuelle est 60 minutes.

Le secret possède une valeur locale de démonstration et doit être remplacé par `PADEL_JWT_SECRET` dans un environnement réel.

7.7. Chaîne de sécurité HTTP

Spring Security désactive CSRF, le formulaire de connexion et HTTP Basic, puis configure une politique stateless. CORS est activé. Le filtre JWT s'exécute avant `UsernamePasswordAuthenticationFilter`.

Les ressources publiques comprennent l'authentification, l'inscription, la santé, la présentation de démonstration, les sites, les terrains, les paramètres métier, OpenAPI, Swagger et la route de console H2.

Les routes `/api/admin/**` exigent le rôle ADMIN. Les routes de membre, disponibilités, matchs, participations et dettes exigent le rôle JOUEUR. Les contrôles de méthode précisent ensuite le propriétaire ou le périmètre.

7.8. Autorisation joueur

`JoueurAuthorizationService` recharge le membre à partir du sujet du JWT et exige qu'il soit actif. Il contrôle :

- l'égalité entre le matricule demandé et celui du token ;
- la propriété d'une participation ;
- la responsabilité d'une dette ;

- la qualité d'organisateur du match.

Pour un identifiant inexistant, certains contrôles laissent l'appel atteindre le service afin que celui-ci retourne un 404 plutôt qu'un 403. Pour une ressource existante appartenant à un autre joueur, l'accès est refusé.

7.9. Autorisation administrateur

`AdminAuthorizationService` recharge l'administrateur actif. Un GLOBAL accède à tous les sites et aux traitements. Un SITE doit fournir son propre `siteId` et ne peut pas créer de fermeture globale.

La sécurité est donc appliquée à deux niveaux : rôle général dans `SecurityConfig`, puis périmètre métier dans `@PreAuthorize`.

7.10. Session frontend

Le JWT et les informations d'identité sont stockés dans `localStorage`. Au chargement, une session sans date d'expiration ou déjà expirée est supprimée. Les changements de stockage entre onglets sont écoutés.

Ce mécanisme améliore l'expérience de démonstration mais n'est pas une session serveur. Une déconnexion efface le stockage local sans révoquer le JWT déjà émis.

7.11. CORS et exposition locale

CORS autorise uniquement `http://localhost:4200`, les méthodes GET, POST, PUT, DELETE et OPTIONS, tous les headers et aucun credential de cookie. Le frontend utilise un bearer token et non un cookie de session.

Le proxy Angular constitue le chemin recommandé en développement. Les communications locales sont en HTTP ; le dépôt ne fournit pas de terminaison TLS de production.

7.12. Points de durcissement identifiés dans le code

Les éléments suivants sont acceptables pour une démonstration mais doivent être revus avant une production :

- secret JWT de repli présent dans la configuration ;
- stockage du JWT dans `localStorage` ;
- absence de refresh token, de révocation et de liste de sessions ;
- absence de claims `iss`, `aud` et `jti` ;
- absence de limitation du nombre de tentatives de connexion ;
- comptes et mots de passe de démonstration exposés lorsque le mode démo est actif ;
- console H2 autorisée dans la chaîne de sécurité ;
- HTTP local sans configuration TLS ;
- seeder PostgreSQL de démonstration actif pour tout démarrage du profil `postgres`.

8. Tests

8.1. Organisation backend

Le backend contient 75 classes de test et 349 méthodes annotées `@Test`. Deux classes suffixées `ITest` regroupent 12 tests PostgreSQL/Testcontainers et sont exclues du cycle Maven par défaut.

Zone	Fichiers de test	Méthodes <code>@Test</code>
Configuration	4	11

Zone	Fichiers de test	Méthodes @Test
Controllers et erreurs HTTP	22	74
Intégration	6	18
Repositories	13	32
Schedulers	2	2
Sécurité	3	24
Services métier	24	187
Smoke de l'application	1	1
Total	75	349

Le cycle `test` exécute 337 méthodes et exclut les 12 méthodes `ITest`. Le profil Maven `postgresql-integration` désactive Surefire et lance uniquement les `*ITest.java` avec Failsafe.

8.2. Couverture backend visible

Les tests couvrent notamment :

- controllers, statuts HTTP, validation et autorisation ;
- services de création, disponibilité, participation, invitation, paiement, dette, fermeture et statistiques ;
- repositories et données de démonstration ;
- génération, validation, expiration et filtrage JWT ;
- gardes de périmètre joueur et administrateur ;
- migration Liquibase ;
- flux d'intégration métier ;
- concurrence sur H2 et PostgreSQL ;
- droits distincts des rôles PostgreSQL ;
- traitements planifiés et compteurs retournés.

Les tests PostgreSQL utilisent `postgres:16-alpine` avec Testcontainers. Ils vérifient, entre autres, le blocage par verrou pessimiste, les doubles paiements concurrents, les inscriptions concurrentes, les traitements simultanés et les droits `padel_migration`, `padel_app` et `padel_readonly`.

8.3. Configuration de test backend

Le profil de test utilise une base H2 en mémoire distincte par contexte grâce à `${random.uuid}`. Les schedulers sont désactivés afin que les tests temporels restent déterministes. Liquibase crée le schéma avec le contexte `test`; les données de démonstration ne sont chargées que par les tests qui appellent explicitement `@Sql("/data.sql")`.

Commandes prévues par le projet :

- Tests H2 et unitaires : `./mvnw test` ou `mvnw.cmd test` sous Windows.
- Intégration PostgreSQL : `./mvnw -Ppostgresql-integration clean verify` avec Docker disponible.

Le code analysé contient les tests et leur séparation. Le présent document ne transforme pas leur simple présence en preuve automatique de réussite ; GitHub Actions est configuré pour les exécuter à chaque contrôle qualité.

8.4. Tests frontend

Le frontend contient 55 fichiers `*.spec.ts`. Une exécution réalisée sur le commit de référence avec npm 11.6.0 a produit 344 tests réussis sur 344 et 55 fichiers réussis sur 55.

La suite couvre les composants, les routes, les guards, l'interceptor, le contexte d'authentification, les services API, les façades, les messages d'erreur et les principaux parcours d'écran.

Commande : `npm run test -- --watch=false`.

8.5. Compilation frontend

La compilation de production `npm run build` a réussi sur le commit analysé. Le bundle initial produit par cette exécution mesurait 386,14 kB brut, sous le seuil d'avertissement de 500 kB déclaré dans `angular.json`. Les pages principales apparaissent comme chunks paresseux distincts.

8.6. Cypress

Deux suites Cypress sont présentes :

- `cypress/e2e/happy-flows.cy.ts` : 9 scénarios avec interceptions HTTP ;
- `cypress/full-stack/full-stack-happy-flow.cy.ts` : 2 scénarios utilisant réellement Angular, Spring Boot et H2.

Les scénarios simulés couvrent l'accueil, les connexions, le solde, les disponibilités, les matchs publics, les invitations, les dettes, les fermetures et les statistiques. Le scénario full-stack couvre le chargement de la présentation et le parcours connexion → disponibilités → création → réservations.

Commandes : `npm run cypress:run` et `npm run cypress:run:fullstack`.

8.7. GitHub Actions

Le workflow `project-quality-gates.yml` définit cinq jobs :

- tests backend H2 ;
- tests d'intégration PostgreSQL avec Docker ;
- démarrage du backend et contrôle de `/api/health` ;
- installation, build et tests frontend ;
- scénarios Cypress simulés puis full-stack.

Il s'exécute sur les pull requests vers `main` et sur les pushes vers `main`. Les permissions sont limitées à la lecture du contenu et les exécutions concurrentes d'une même référence sont annulées au profit de la plus récente.

9. Organisation Git et méthode de travail

9.1. Dépôt de référence

Le dépôt est <https://github.com/MariusNMU/Paddel-Marius>. La branche par défaut est `main`. L'instantané documenté est le commit :

`9ce8b8f1cfd5679abb38e02f01cc99bad3a53b23`

9.2. Historique observable

À ce commit, la branche `main` contient 354 commits. Parmi leurs sujets, 139 commencent explicitement par `Merge pull request #...`. L'historique montre donc une intégration répétée de branches par pull request, y compris pour les évolutions métier, les tests, la documentation, la sécurité et les mises à jour techniques.

9.3. Contrôles de qualité versionnés

Le dépôt versionne :

- les wrappers Maven Windows et Unix ;
- `package-lock.json` et la version npm attendue ;
- les migrations et scripts DB ;
- les tests backend, frontend et Cypress ;
- le workflow GitHub Actions ;
- les documents d'architecture, d'exploitation et de démonstration ;
- des `.gitignore` distincts à la racine, dans le backend et dans le frontend.

Les dossiers générés (`target`, `dist`, `node_modules`, cache Angular, captures et vidéos Cypress) sont exclus de Git.

10. Exploitation du projet

10.1. Prérequis

Outil	Version ou condition issue du code
Java	21
Node.js	<code>^22.12.0</code> ou <code>^24.0.0</code>
npm	11.6.0
Docker	Facultatif pour l'exécution H2 ; requis pour PostgreSQL et les <code>ITest</code>
Navigateur	Accès à Angular sur le port 4200

Les wrappers Maven fournis évitent d'exiger une installation Maven globale.

10.2. Démarrage H2 par défaut

Depuis la racine du dépôt :

- Ouvrir un terminal dans `backend`.
- Sous Windows, exécuter `mvnw.cmd spring-boot:run` ; sous Linux/macOS, `./mvnw spring-boot:run`.
- Vérifier `http://localhost:8080/api/health`.
- Ouvrir Swagger à `http://localhost:8080/swagger-ui.html` si nécessaire.

Le backend crée une base H2 en mémoire, applique Liquibase et charge les données démo. Un redémarrage réinitialise cette base.

10.3. Démarrage du frontend

Dans un second terminal :

- Ouvrir `frontend`.
- Vérifier que npm 11.6.0 est utilisé.
- Exécuter `npm ci`.
- Exécuter `npm start`.
- Ouvrir `http://localhost:4200`.

Le script `start` lance `ng serve` sur le port 4200 avec `proxy.conf.json`.

10.4. Démarrage PostgreSQL

Pour utiliser le profil PostgreSQL :

- À la racine, exécuter `docker compose up -d postgres`.
- Attendre que le conteneur soit sain.
- Dans `backend`, lancer `mvnw.cmd spring-boot:run -Dspring-boot.run.profiles=postgres` sous Windows, ou l'équivalent `./mvnw` sous Linux/macOS.
- Démarrer ensuite le frontend normalement.

Le volume `padel_postgres_data` conserve les données entre les redémarrages. `docker compose down` arrête le service sans supprimer le volume. `docker compose down -v` supprime le volume et force une réinitialisation complète au prochain démarrage.

10.5. Variables d'environnement utiles

Variable	Usage
<code>PADEL_JWT_SECRET</code>	Remplacer le secret JWT local
<code>SPRING_DATASOURCE_URL</code>	URL JDBC du backend PostgreSQL
<code>SPRING_DATASOURCE_USERNAME</code>	Utilisateur applicatif
<code>SPRING_DATASOURCE_PASSWORD</code>	Mot de passe applicatif
<code>SPRING_LIQUIBASE_URL</code>	URL utilisée par Liquibase
<code>SPRING_LIQUIBASE_USER</code>	Utilisateur de migration
<code>SPRING_LIQUIBASE_PASSWORD</code>	Mot de passe de migration
<code>PADEL_DEMO_ENABLED</code>	Activer ou désactiver la présentation démo
<code>PADEL_VEILLE_PLANIFICATION_ACTIVE</code>	Activer le traitement J-1
<code>PADEL_VEILLE_DELAI_INITIAL_MS</code>	Délai initial J-1
<code>PADEL_VEILLE_INTERVALLE_MS</code>	Intervalle J-1
<code>PADEL_ECHEANCES_PLANIFICATION_ACTIVE</code>	Activer le traitement d'échéance
<code>PADEL_ECHEANCES_DELAI_INITIAL_MS</code>	Délai initial d'échéance
<code>PADEL_ECHEANCES_INTERVALLE_MS</code>	Intervalle d'échéance

10.6. Comptes de démonstration publiés

Lorsque le mode démo est actif, l'accueil publie les comptes suivants.

Type	Identifiant	Mot de passe	Particularité
Joueur	<code>G1001</code>	<code>password</code>	GLOBAL actif
Joueur	<code>G1002</code>	<code>password</code>	Dette ouverte et pénalité active
Joueur	<code>S1001</code>	<code>password</code>	SITE Bruxelles
Joueur	<code>S1002</code>	<code>password</code>	SITE Namur
Joueur	<code>L1001</code>	<code>password</code>	LIBRE actif
Joueur	<code>L1002</code>	<code>password</code>	LIBRE actif
Joueur	<code>G9999</code>	<code>password</code>	Inactif, utilisé pour le refus
Admin	<code>admin-global</code>	<code>secret</code>	GLOBAL
Admin	<code>admin-bruxelles</code>	<code>secret-site</code>	SITE Bruxelles

Ces identifiants sont exclusivement destinés à la démonstration locale.

10.7. Données de démonstration

Le seed H2 crée trois sites, dont Liège inactif ; sept terrains, dont un terrain de Bruxelles inactif ; des horaires pour l'année courante et la suivante ; deux fermetures futures ; sept membres ; quatre administrateurs ; quatre matchs ; neuf participations ; une dette ; une pénalité et huit paiements.

Les dates relatives évitent qu'un jeu de données figé ne devienne immédiatement inutilisable. Le seeder PostgreSQL reproduit le même objectif avec des `INSERT ... ON CONFLICT` et synchronise les séquences après les identifiants explicites.

10.8. Commandes de validation

Depuis `backend` :

- `mvnw.cmd test` sous Windows ou `./mvnw test` sous Linux/macOS.
- Avec Docker : `mvnw.cmd -Ppostgresql-integration clean verify` ou l'équivalent Unix.

Depuis `frontend` :

- `npm ci`.
- `npm run build`.
- `npm run test -- --watch=false`.
- `npm run cypress:run` pour les scénarios simulés.
- `npm run cypress:run:fullstack` pour le parcours réel.

10.9. Ordre de démonstration conseillé

- Démarrer le backend H2 et vérifier `/api/health`.
- Démarrer Angular et ouvrir l'accueil.
- Se connecter comme joueur GLOBAL actif.
- Consulter les disponibilités, créer un match et payer la participation organisateur.
- Montrer une invitation ou rejoindre un match public.
- Consulter les réservations, le solde, l'historique et les dettes.
- Se connecter comme administrateur GLOBAL.
- Ouvrir le planning hebdomadaire, naviguer entre deux semaines et montrer une réservation, une fermeture et un terrain inactif, puis présenter les membres et les statistiques.
- Exécuter les traitements manuels si le jeu de données s'y prête.
- Créer une fermeture sur une date de test et montrer les compteurs de remboursement.

11. Limites du MVP

11.1. Limites fonctionnelles

Le paiement utilise uniquement un crédit interne. Il n'existe ni recharge, ni paiement bancaire, ni remboursement vers une carte. Le paiement partiel d'une dette n'est pas accepté.

Les joueurs ne disposent pas d'une fonction d'annulation volontaire d'un match, de modification du profil ou de récupération de mot de passe. Le projet ne contient pas de messagerie, d'e-mail, de notification push ni de mise à jour temps réel. Le badge d'invitation repose sur des appels API déclenchés par le frontend.

L'administration ne fournit pas un CRUD complet des sites, terrains, horaires, membres ou comptes administrateurs. La gestion des pénalités ne possède pas d'écran dédié.

11.2. Limites de sécurité

Le système ne fournit ni refresh token, ni révocation, ni session unique côté serveur. La déconnexion locale ne rend pas un JWT copié inutilisable avant son expiration.

Le JWT est stocké dans `localStorage`. Le service valide la signature et l'expiration, mais ne vérifie pas d'émetteur, d'audience ou d'identifiant de token. Aucune limitation de tentatives ou protection anti-brute-force n'est implémentée.

Le secret de repli, les comptes de démo et l'autorisation de la console H2 conviennent à un environnement étudiant contrôlé, pas à un déploiement public. La configuration TLS et la gestion centralisée de secrets ne font pas partie du dépôt.

11.3. Limites de persistance et d'exploitation

H2 est volatile. PostgreSQL conserve les données dans un volume, mais le seeder de démonstration s'exécute à chaque démarrage du profil et effectue des mises à jour idempotentes sur ses identifiants de référence.

Le schéma ne contient pas de contraintes SQL pour la capacité maximale, le chevauchement, les dates de début/fin ou les valeurs d'énumération. Ces règles dépendent du backend.

La génération de matricule repose sur le plus grand matricule existant puis +1. La contrainte unique protège la base, mais le service ne comporte pas de mécanisme de reprise spécifique si deux inscriptions concurrentes génèrent la même valeur.

11.4. Limites de performance et d'échelle

Les listes ne sont pas paginées. Plusieurs statistiques et contrôles chargent des collections puis les filtrent en mémoire. Cette stratégie reste lisible pour le MVP et les données de démonstration, mais devrait être remplacée par des requêtes agrégées et paginées pour un volume important.

Les schedulers sont conçus pour une instance de démonstration. Les verrous de base sécurisent les lignes traitées, mais le dépôt ne configure ni orchestration distribuée, ni leader election, ni supervision externe.

11.5. Limites de validation de session frontend

L'expiration est contrôlée lors de l'initialisation et des événements de stockage. Une page laissée ouverte ne possède pas de minuterie qui déconnecte exactement à l'échéance ; les guards peuvent donc conserver l'état « connecté » jusqu'au prochain rechargement ou jusqu'à une réponse 401 du backend.

12. Brève conclusion

Le code actuel couvre les acteurs joueur et administrateur, les catégories de membre, les disponibilités multi-sites, les matchs privés et publics, les invitations, le crédit interne, les dettes, les pénalités, les fermetures avec remboursement, les statistiques et les traitements temporels.

L'architecture sépare clairement interface, état d'écran, appels HTTP, logique métier et persistance. Les DTO limitent l'exposition du modèle, les verrous protègent les écritures sensibles et les rôles PostgreSQL séparent migrations, application et lecture seule.

Les limites identifiées confirment la nature du livrable : il est adapté à un projet pédagogique et à une démonstration métier, mais nécessite un durcissement de sécurité, une intégration de paiement

réelle, une administration plus complète et des adaptations d'échelle avant une utilisation de production.

À la suite des remarques lors de la première session, j'ai revu en profondeur mon projet afin d'essayer de le rendre plus robuste. Parmi les principales améliorations, j'ai fait :

- la suppression des données codées en dur ;
- le renforcement de la sécurité avec Spring Security, filtre JWT, gestion des rôles et contrôle des sessions
- une restructuration du frontend Angular avec des façades séparant l'interface, l'état et la logique applicative ;
- nombreuses améliorations de l'UX, des libellés et des droits d'action, ainsi que développement des tests, de la CI et de la documentation.

Le présent cahier des charges constitue donc la photographie fonctionnelle et technique du [main](#) au 4 août 2026. Toute évolution ultérieure devra mettre à jour le commit de référence et les chapitres concernés afin d'éviter un nouvel écart entre documentation et code.

Annexes

Annexe A : Catalogue des endpoints REST

Méthode	Endpoint	Accès	Fonction
POST	/api/auth/joueur	Public	Authentifier un joueur
POST	/api/auth/admin	Public	Authentifier un administrateur
POST	/api/membres/inscription	Public	Inscrire un membre
GET	/api/health	Public	Vérifier le démarrage du backend
GET	/api/demo/presentation	Public, si démo active	Charger les informations de démonstration
GET	/api/sites	Public	Lister les sites actifs
GET	/api/terrains	Public	Lister les terrains et sites actifs
GET	/api/parametres-metier	Public	Exposer les constantes métier
GET	/api/disponibilites	Joueur actif	Calculer les créneaux d'un site et d'une date
POST	/api/matches	Joueur propriétaire	Créer un match
GET	/api/matches/publics	Joueur actif	Lister les matchs publics et l'éligibilité
POST	/api/matches/{matchId}/participants/public/payer	Joueur propriétaire	Rejoindre et payer un match public
POST	/api/matches/{matchId}/invitations/primees	Organisateur	Inviter un joueur
GET	/api/membres/{matricule}/invitations/recues	Joueur propriétaire	Lister ses invitations
GET	/api/membres/{matricule}/invitations/recues/count	Joueur propriétaire	Compter ses invitations
POST	/api/participations/{participationId}/decliner	Joueur propriétaire	Décliner une invitation
POST	/api/participations/{participationId}/paiements	Joueur propriétaire	Payer une participation

Méthode	Endpoint	Accès	Fonction
GET	/api/membres/{matricule}/paiements	Joueur propriétaire	Consulter l'historique
GET	/api/membres/{matricule}/reservations	Joueur propriétaire	Consulter ses réservations
GET	/api/membres/{matricule}/solde	Joueur propriétaire	Consulter son solde
GET	/api/membres/{matricule}/dettes/ouvertes	Joueur propriétaire	Lister ses dettes ouvertes
POST	/api/dettes/{detteId}/paiements	Responsable de la dette	Régler une dette
GET	/api/admin/sites	Admin GLOBAL	Lister tous les sites, actifs ou non
GET	/api/admin/membres	Admin selon périmètre	Lister tous les membres ou ceux d'un site
POST	/api/admin/fermetures	Admin selon portée	Créer une fermeture et annuler/rembourser
GET	/api/admin/etat-operationnel	Admin du site ou GLOBAL	Consulter terrains et matchs d'une date
GET	/api/admin/etat-operationnel/semaine	Admin du site ou GLOBAL	Consulter l'occupation du lundi au dimanche
GET	/api/admin/statistiques	Admin selon périmètre	Calculer les statistiques d'une période
POST	/api/admin/matches/traitement-veille	Admin GLOBAL	Lancer manuellement le traitement J-1
POST	/api/admin/matches/traitement-echeance	Admin GLOBAL	Lancer manuellement le traitement d'échéance

OpenAPI est exposé séparément à </v3/api-docs> et Swagger UI à </swagger-ui.html>.

Annexe B : Routes Angular

Route	Accès frontend	Page
/accueil	Public	Présentation et comptes démo
/joueur	Public	Connexion joueur
/inscription-joueur	Public	Inscription
/joueur/disponibilites	joueurGuard	Organiser un match
/joueur/creer-match	joueurGuard	Créer le match et inviter
/joueur/matches-publics	joueurGuard	Rejoindre et payer
/joueur/mes-reservations	joueurGuard	Réservations et paiement
/joueur/invitations-recues	joueurGuard	Confirmer ou décliner
/joueur/mes-dettes	joueurGuard	Dettes ouvertes
/joueur/historique-transactions	joueurGuard	Historique des paiements
/joueur/mon-solde	joueurGuard	Solde crédit
/admin/login	Public	Connexion administrateur
/admin/dashboard	adminGuard	Tableau de bord
/admin/membres	adminGuard	Liste des membres
/admin/statistiques	adminGuard	Statistiques
/admin/etat-operationnel	adminGuard	Planning hebdomadaire des terrains

Route	Accès frontend	Page
/admin/fermetures	adminGuard	Création de fermetures
/admin/traitement-veille	adminGlobalGuard	Traitement J-1 manuel
/admin/traitement-echeance	adminGlobalGuard	Traitement d'échéance manuel

La route vide redirige vers /accueil et la wildcard fait de même.

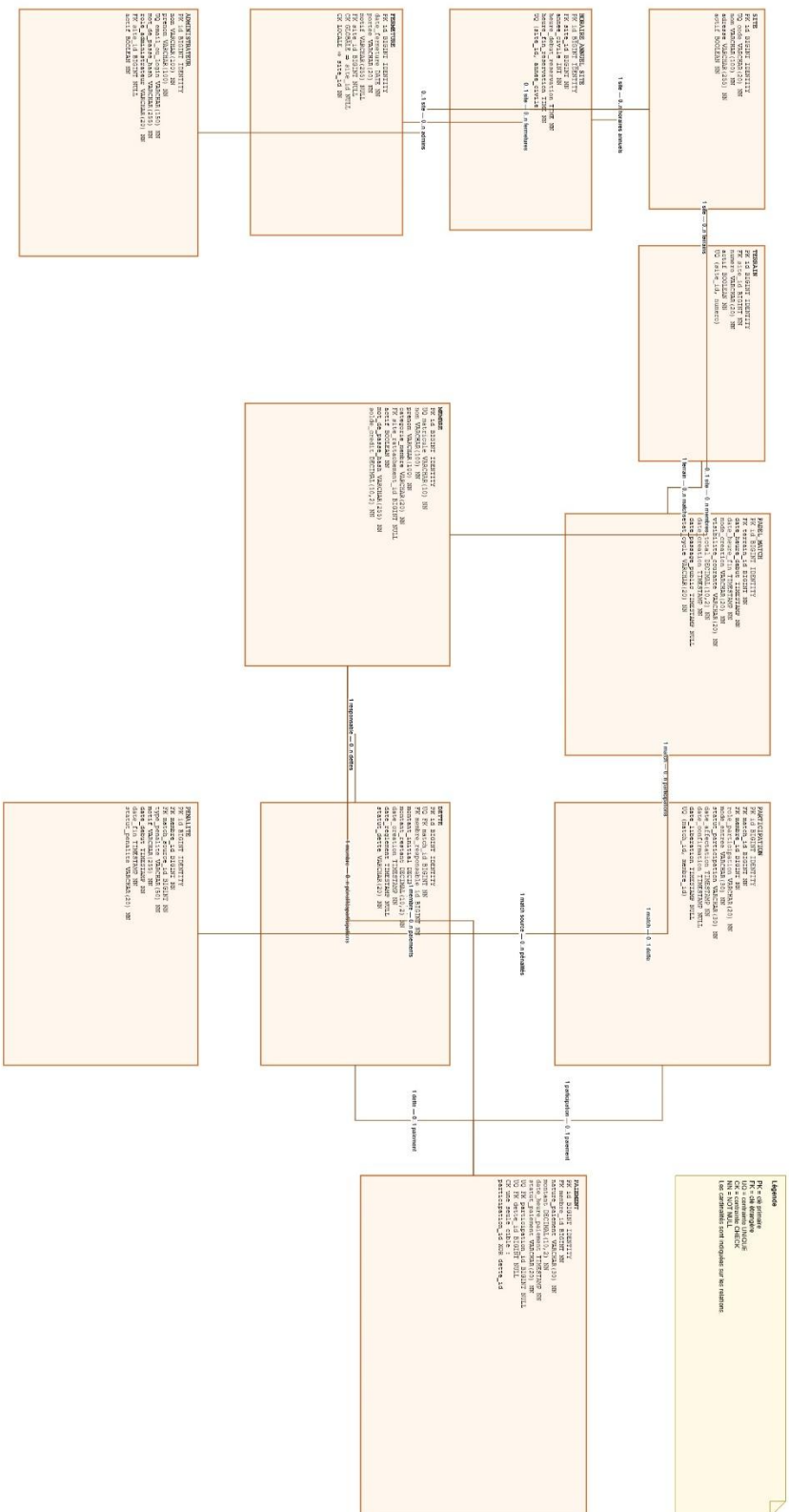
Annexe C : Fichiers de traçabilité essentiels

Sujet	Fichiers de référence dans le dépôt
Versions backend/frontend	backend/pom.xml, frontend/package.json, frontend/angular.json
Règles chiffrées	backend/src/main/java/com/padelMarius/backend/config/ReglesMetier.java
Sécurité HTTP	SecurityConfig.java, JwtAuthenticationFilter.java, JwtService.java
Autorisations	AdminAuthorizationService.java, JoueurAuthorizationService.java
Création et disponibilités	MatchCreationService.java, DisponibiliteService.java, ReglesReservationMembreService.java
Participation et invitations	ParticipationService.java, InvitationPriveeService.java, MatchPublicService.java
Paiements et dettes	PaieementService.java, DetteService.java
Cycle temporel	TraitementVeilleService.java, TraitementEcheanceService.java, dossier scheduler
Fermetures et remboursements	AdminFermetureService.java
Administration	EtatOperationnelAdminService.java, StatistiquesAdminService.java, MembreAdminService.java
Schéma	db/changelog/changes/001-create-initial-schema.sql
Données H2	backend/src/main/resources/data.sql
PostgreSQL	docker-compose.yml, 01-create-users-and-rights.sql, application-postgres.properties
Routes Angular	frontend/src/app/app.routes.ts
Session et JWT frontend	auth-context.service.ts, admin-auth.interceptor.ts, dossier guards
CI	.github/workflows/project-quality-gates.yml

Annexe D : Contrôle de version du présent document

Champ	Valeur
Dépôt	MariusNMU/Padel-Marius
Branche	main
Commit	9ce8b8f1cfd5679abb38e02f01cc99bad3a53b23
Date du snapshot	7 août 2026
Spring Boot	4.0.6
Angular	21.2.19
Java	21
Base par défaut	H2 en mémoire
Base optionnelle	PostgreSQL 16 via Docker

Schéma relationnel SQL — 11 tables

[illegible]